

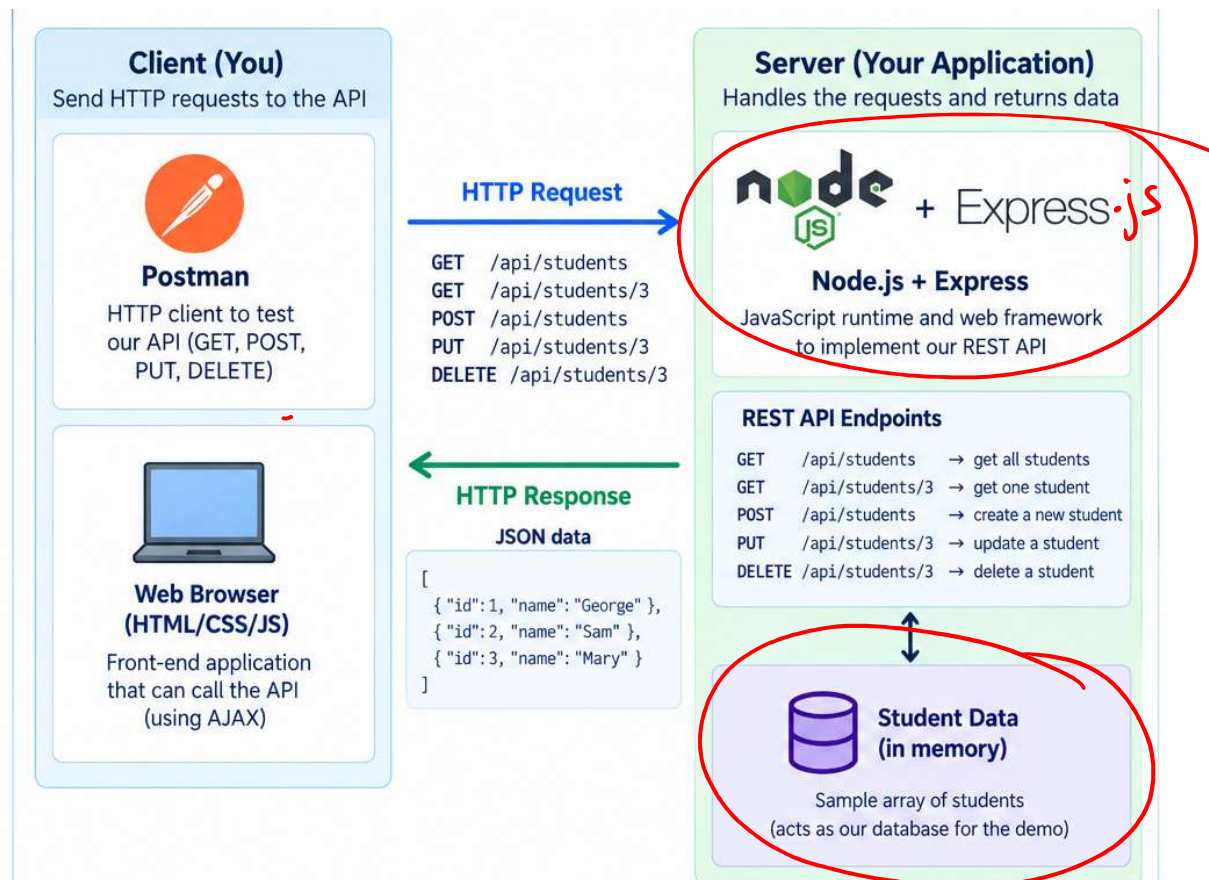
Announcements

- Maven Activity and Project 1 Progress Report is Due this Sunday
- You should now start working on the Project 1 Requirements Assignment
- REST assignment was out last Thursday - This is an individual assignment. Due Next Sunday.
- Extra Credit Opportunity in today's class

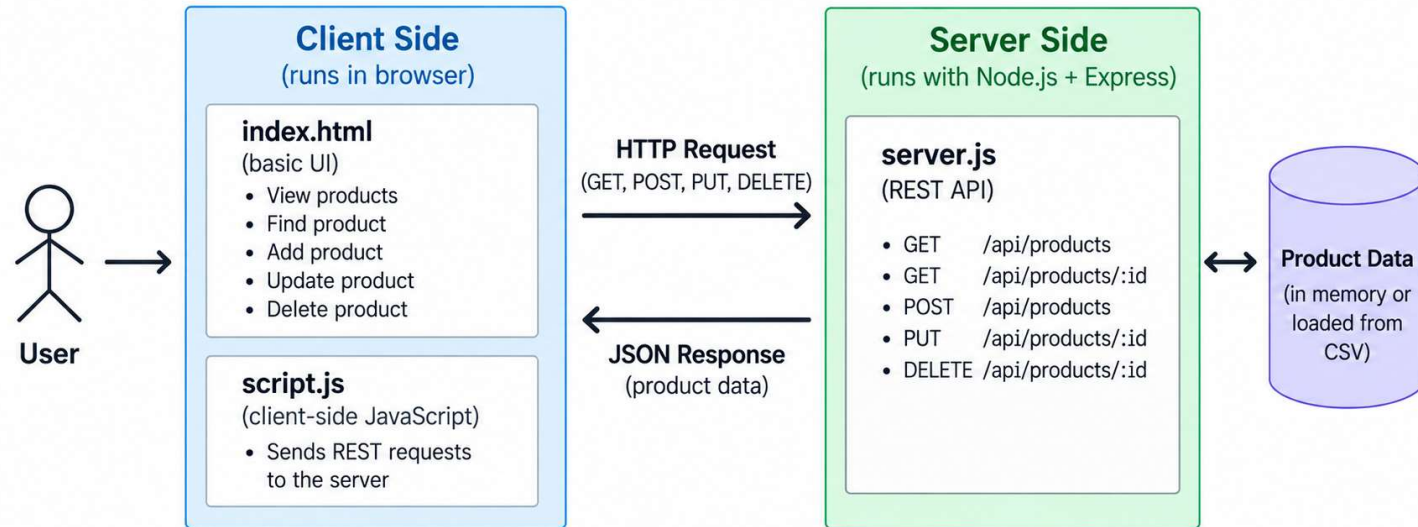
DEMO TIME!!

Create a basic web app using REST API, VS Code IDE, Node.JS, Express and Postman. Build the server and API and postman to test it.

jsonbin.io
Postman
Testing



REST ASSIGNMENT



What you need to demonstrate

- A simple UI — it does **not** need to be polished.
- script.js runs in the browser and sends the REST requests.
- server.js runs Node.js/Express and implements the REST endpoints.
- You may initialize the server with the provided CSV data.
- **POST must still be demonstrated** by adding a product through the client.
- Demonstrate GET, POST, PUT, and DELETE **through your client application, not Postman.**

CS3300 Introduction to Software Engineering

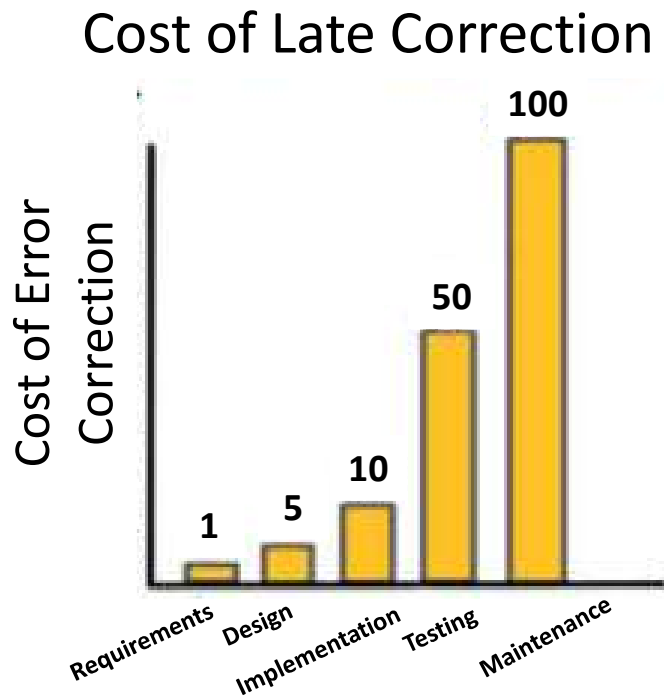
Lecture 07: Requirements Engineering

Dr. Nimisha Roy ▶ nroy9@gatech.edu

Requirements Engineering (RE)

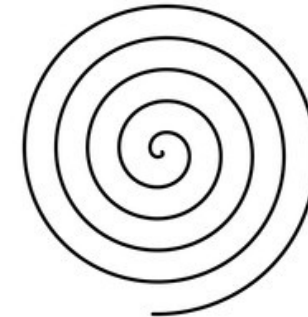


RE is the process of establishing the needs of stakeholders that are to be solved by software



Management

Elicitation



Analysis

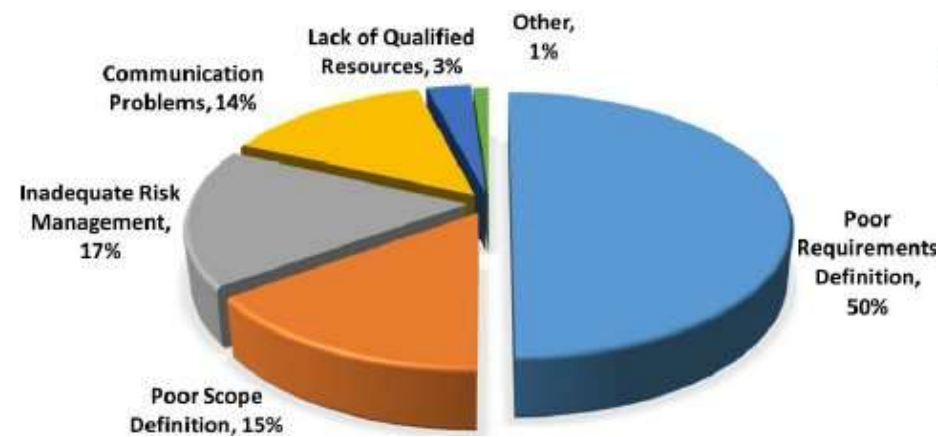
Validation

Specification

SRS
Software Requirements
Specification

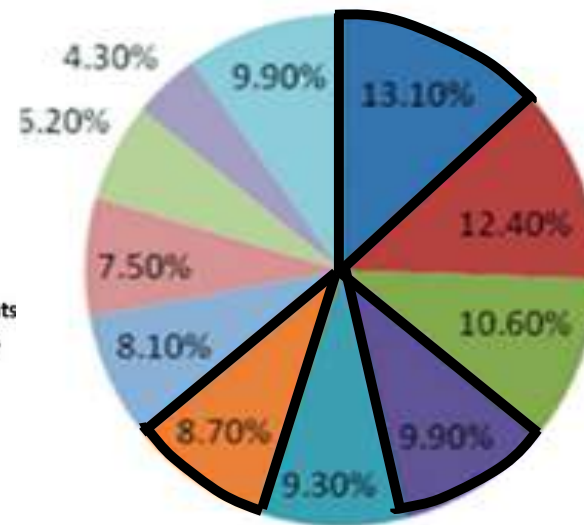


Software Failures due to RE



Role of Requirements in Software Project Failures (Agile)

Source: Abdou, T., Kamthan, P., & Shahmir, N. (2014). *User Stories for Agile Business: INVEST, Carefully!.* Social Media and Publicity, 141.



Role of Requirements in Software Project Failures.

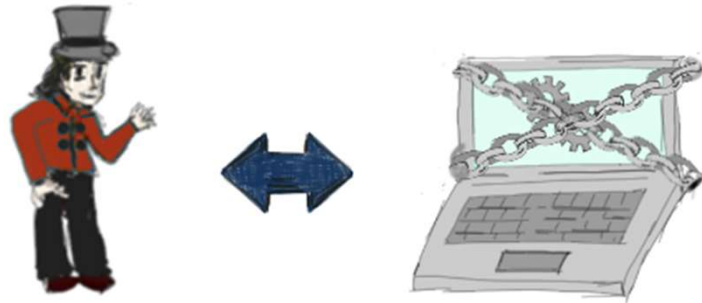
Source: Hussain, A., Mkpojiogu, E. O., & Kamal, F. M. (2016). *The role of requirements in the success or failure of software projects.* International Review of Management and Marketing, 6(7S), 306-311

- Incomplete Requirements
- Lack of user involvement
- Lack of Resources
- Unrealistic Expectations
- Lack of Executive Support
- Changing Requirements
- Lack of planning
- Not needed any longer
- Lack of IT Management
- Technology Illiteracy
- Others

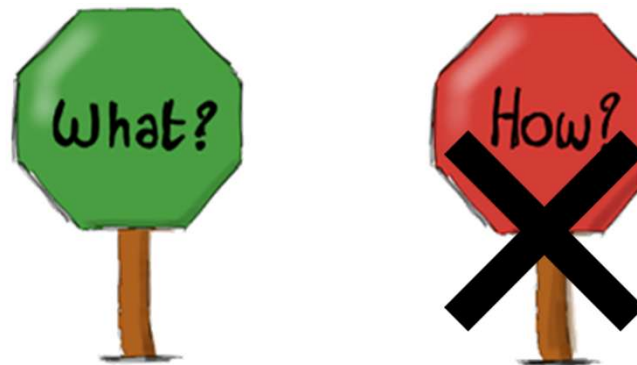
Requirements Engineering (RE)



= =

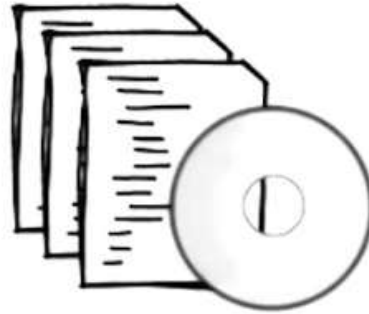


=> Software Requirements Specification (SRS)



Software Intensive Systems

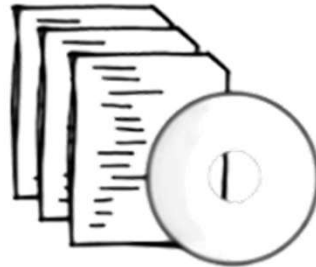
Software



Software Intensive System = Software + Hardware + Context



=



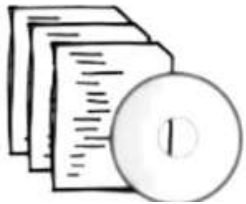
+



+



Software Quality



Software runs on some hardware and is developed for a purpose that is related to human activities

$$\text{Quality} \neq f(\text{stack of papers and CD})$$

$$\text{Quality} = f(\text{stack of papers and CD}, \text{target with arrow})$$



RE is mostly about identifying the purpose

Identifying Purpose = Defining Requirements



Extremely Hard Task

- Sheer Complexity of the purpose/requirements
- “Often, people don’t know what they want until you show it to them” – Steve Jobs
- Changing requirements
- Multiple stakeholders with conflicting requirements

Definition of Requirements Engineering

Requirements Engineering (RE) is a set of activities concerned with identifying and communicating the purpose of software-intensive system, and the context in which it will be used. Hence, RE acts as the bridge between the real-world needs of users, customers, and other constituencies affected by a software system, and the capabilities and opportunities afforded by software – intensive technologies.

Communication is as important as analysis

Quality means fitness-for-purpose. Cannot say anything about quality unless you understand the purpose

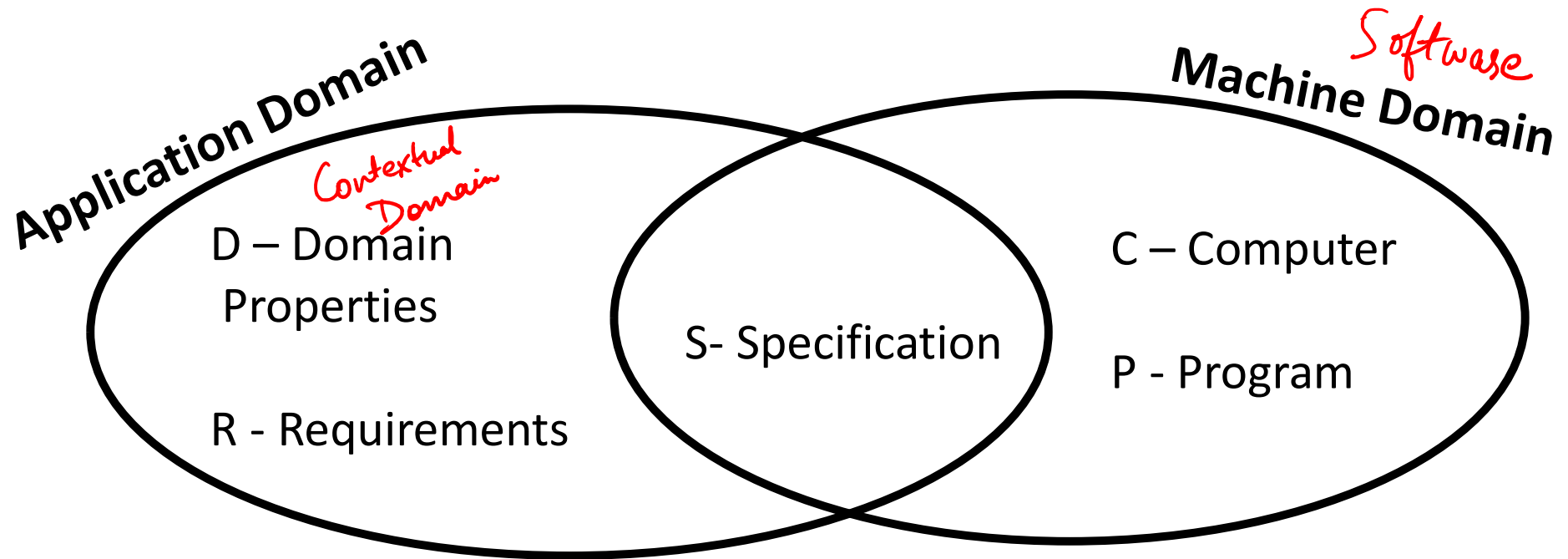
Needed to identify all stakeholders – not just the customer or user

Designers need to know how and where the system will be used

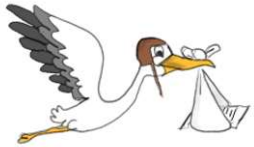
Requirements are partly about what is needed...

... and partly about what is possible

RE is a shared phenomenon between domains



- Machine Domain - hardware/OS/libraries
- Application Domain - world in which software will operate
- Events in real world that machine can detect – buttons pushed
- Actions in real world that machine can cause- image appearing on screen



Requirements Elicitation: Where do Requirements come from?

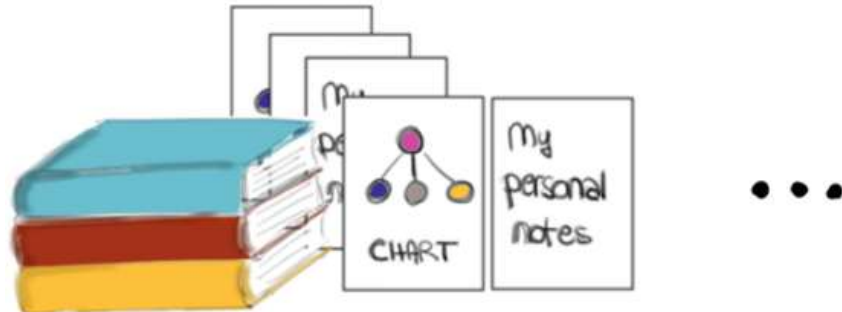
Stakeholders



Application Domain



Documentation



Requirements Elicitation: Other Techniques

Collaborative Techniques - Brainstorming

A software company is developing a new mobile game and invites both developers and gamers for a brainstorming session. During the session, a suggestion for a unique game mode emerges that becomes a central feature of the final product.



Social Approaches - Ethnographic techniques

Analysts spend a day in a busy restaurant to observe how servers use the point-of-sale system. They notice that servers often bypass a time-consuming screen, jotting orders down on paper to enter later, indicating a need for a more efficient user interface.



Requirements Elicitation: Other Techniques

Cognitive techniques - Problem solving methods,
Prototyping, Cognitive Task Analysis

In designing software for air traffic controllers, analysts study the controllers' processes for tracking multiple planes. They find that certain visual cues and alerts can reduce mental strain



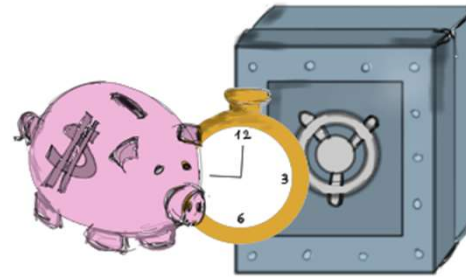
Requirements Elicitation: Functional and Non-functional Requirements

Functional



*How to
quantitatively
measure
requirements.*

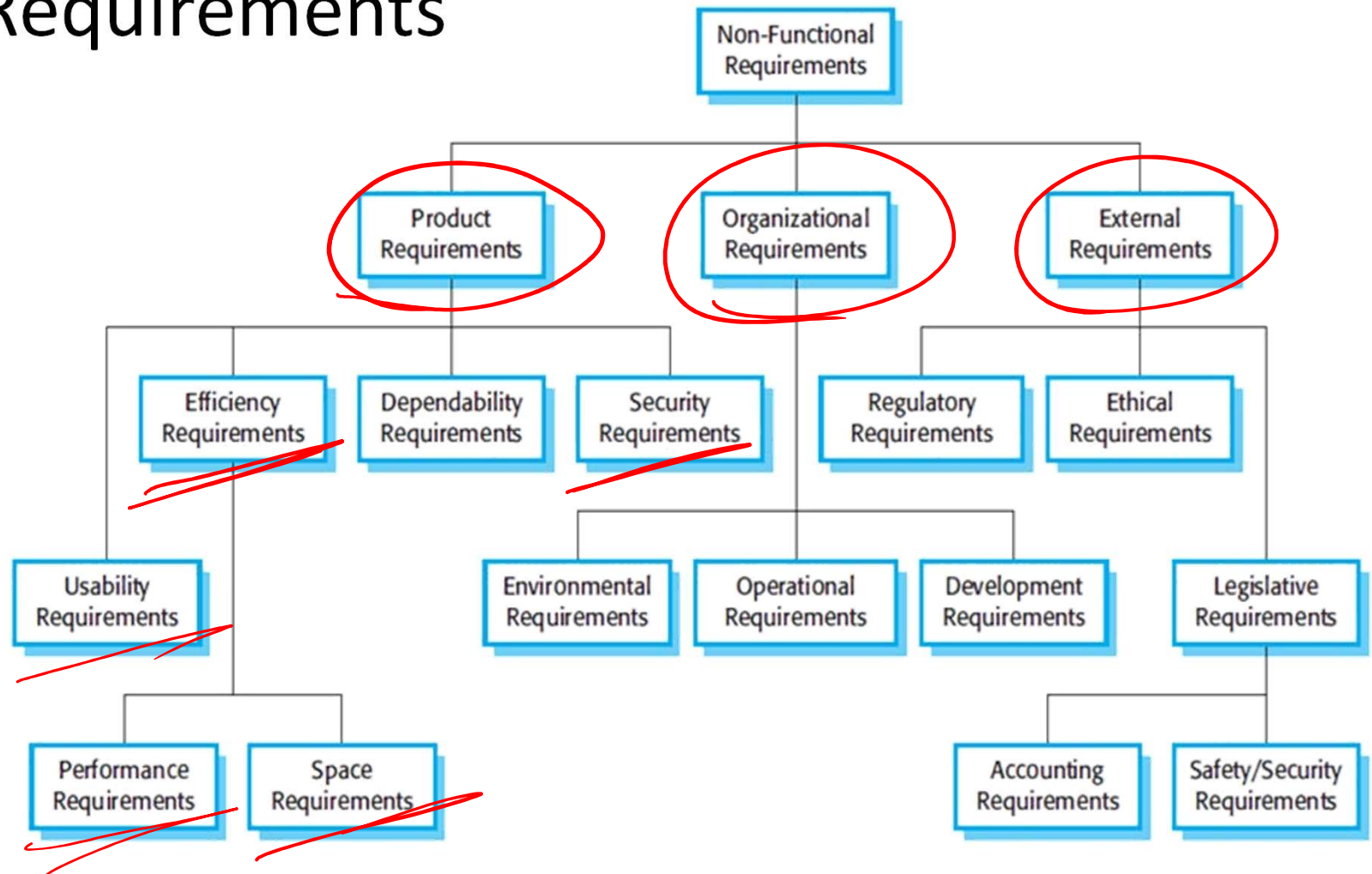
~~Non-Functional~~



Non-Functional requirements: criteria that can be used to judge the operation of a system, rather than specific behaviors. In essence, they describe how the system works, rather than what it does. E.g. security, accuracy, performance, cost, usability, adaptability, interoperability, reusability and so on.

Requirements Elicitation: Functional and Non-functional Requirements

Non-Functional Requirement may be more critical than functional requirement



Requirements should be measurable



Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

An example: MHC-PMS: A patient management system for mental health care

- This case study describes a real system (although that is not its real name) which was used (and may still be used) in a number of UK hospitals, including hospitals in Scotland.
- The system is designed for use in clinics attended by patients suffering from mental health problems and records details of their consultations and conditions.
- It is separate from a more general patient records system as more detailed information has to be maintained and the system has to be set up to generate letters and reports of different types and to help ensure that the laws pertaining to mental health are maintained by staff treating patients.

MHC-PMS: Non functional Requirements

- **Product requirement** The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.
Availability
Reliability
- **Organizational requirement** Users of the MHC-PMS system shall authenticate themselves using their health authority identity card.
Security
- **External requirement:** The system shall implement patient privacy provisions as set out in the country's law.
regulatory

Example Quiz

A software application designed for a hospital is running smoothly on its intended hardware. However, the nurses using the system report that it takes too long to enter patient information, causing delays in patient care. Based on the definition of software quality discussed, what type of requirement might need improvement?

- A) Functional Requirement
- ~~B) Non-Functional Product Requirement~~
- C) Non-functional Organizational Requirement
- D) Non-functional External Requirement

Speed

Software

environment

domain

Software Quality & RE: ICU Monitoring System

System: Real-time patient monitoring system that tracks vital signs and alerts healthcare professionals when readings become abnormal.

Quality depends on its context:

- Software: Process real-time data accurately and generate appropriate alerts.
- Hardware: Reliable sensors and monitors must operate continuously.
- Context: ICU is a high-stakes, 24/7 environment where delayed or inaccurate alerts can have serious consequences.

Software Quality & RE: ICU Monitoring System

Requirements Engineering translates that context into requirements:

- **Functional:** The system shall alert healthcare professionals when a vital sign exceeds its defined safe range. *hardware*
- **Non-functional Speed:** Alerts shall be generated within 1 second of detecting an abnormal reading.
- **Non-functional Reliability:** Downtime shall be less than 0.1%.
- **Integration/constraints:** Must integrate with existing hospital systems and satisfy applicable privacy requirements.

Takeaway: Quality = fitness for purpose → understanding the context helps us define the right requirements.

Requirements Elicitation & Analysis: Traditional Techniques



Background Reading



Hard Data and Samples



Interviews

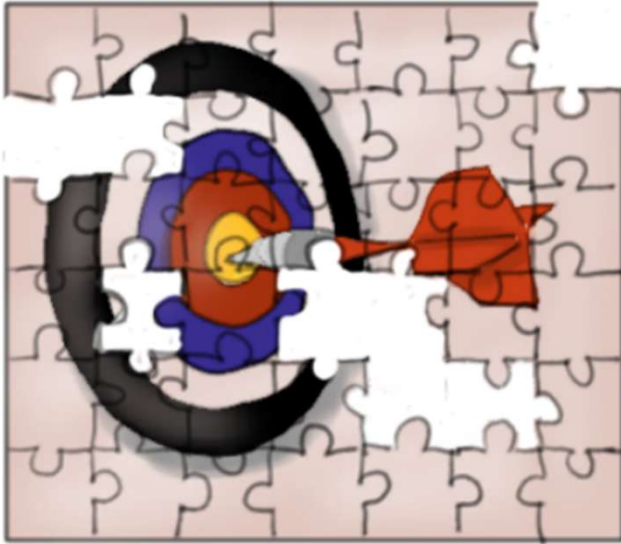


Surveys

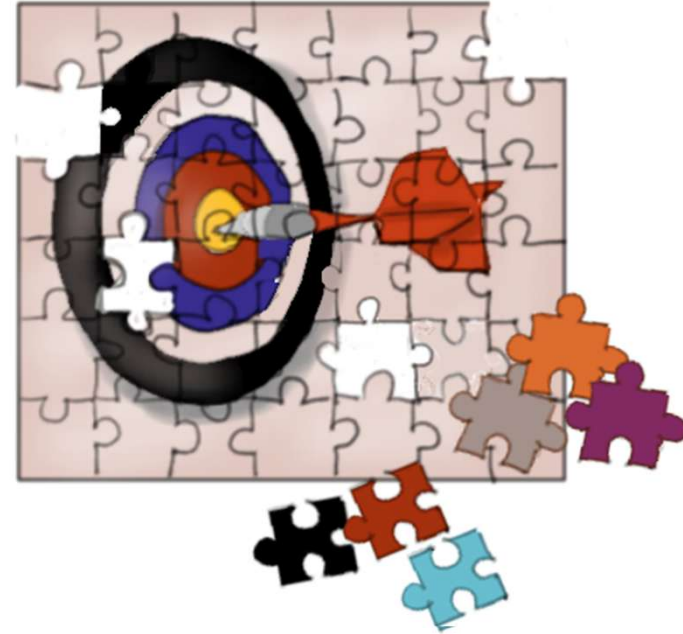


Meetings

Requirements Analysis: Completeness and Pertinence

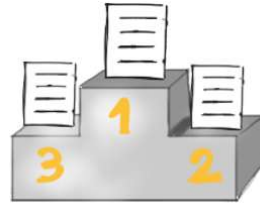


Difficult to identify all requirements,
incomplete requirements collected and
S/W missing functionality



- Relevance of requirements;
- Irrelevant conflicting requirements collected for sake of completeness
- Worse case: completeness issue not solved, irrelevant requirements with information harmful to system

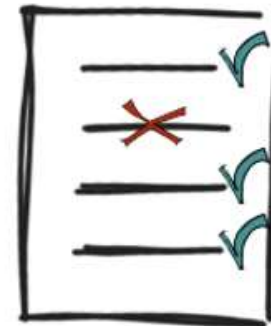
Requirements Analysis: Requirements Prioritization



Limited Resources



=> Inability to satisfy all the requirements



⇒ Need to prioritize them

- Mandatory
 - Nice to have
 - Superfluous
- Create your sprint backlog



Example Quiz: Completeness and Pertinence

A software development team is creating a mobile app for online grocery shopping. They collected several requirements, including the ability to browse items, add items to a cart, make payments, and view order history. However, they also included a requirement to integrate a weather forecast feature that shows the weather in the user's location. Which issue does this extra feature represent?

- A) Lack of Completeness
- ☒ B) Lack of Pertinence
- C) Incomplete Requirements
- D) Redundant Requirements

Requirements Specification: User and System Requirements

User Requirements



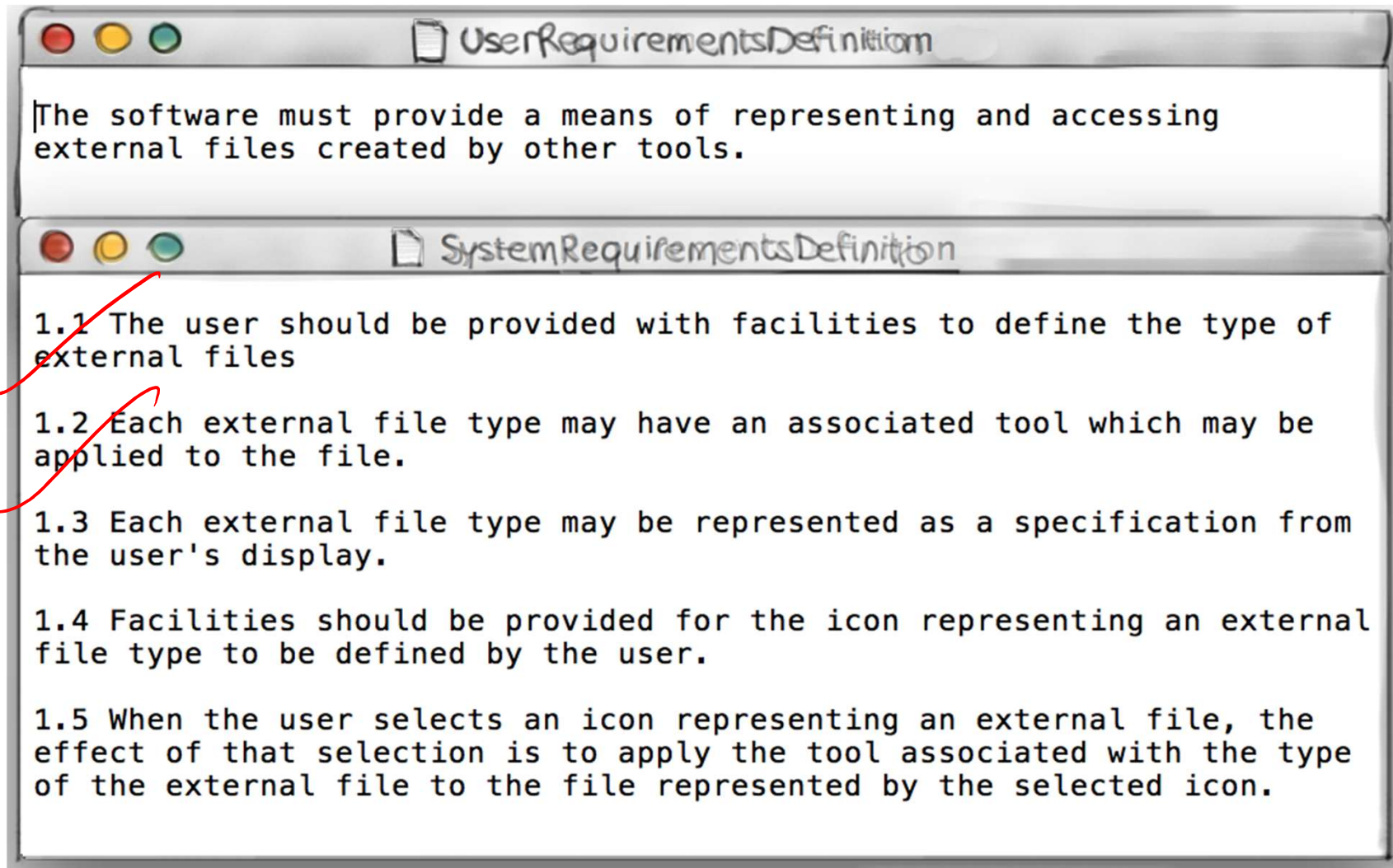
- Written for customers
- Often in natural language, no technical details

System Requirements



- Written for developers
- Detailed functional and non-functional requirements
- Clearly and more rigorously specified

User and System Requirements



Requirements Specification: SRS

- The software requirements specification document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.
- Your project assignment uses the lightweight IEEE format of Software Requirements Specifications Document

Example Quiz

A team is developing a new financial software tool for a bank. After conducting initial interviews, they find that different departments have conflicting requirements: one wants a simplified user interface for quick transactions, while another requires detailed analytics tools that clutter the interface. What should the team focus on next according to the principles of Requirements Engineering?

- A) Begin coding the software to meet the needs of one department.
- B) Develop a prototype incorporating all requested features.
-  C) Conduct further stakeholder analysis to prioritize and clarify requirements.
- D) Decide independently which department's needs are more critical.

Bad → Good Requirement Activity

Is this a good requirement?

"The system should make it easy for students to find professors quickly."

Turn it into measurable and testable requirements?

Bad → Good Requirement Activity

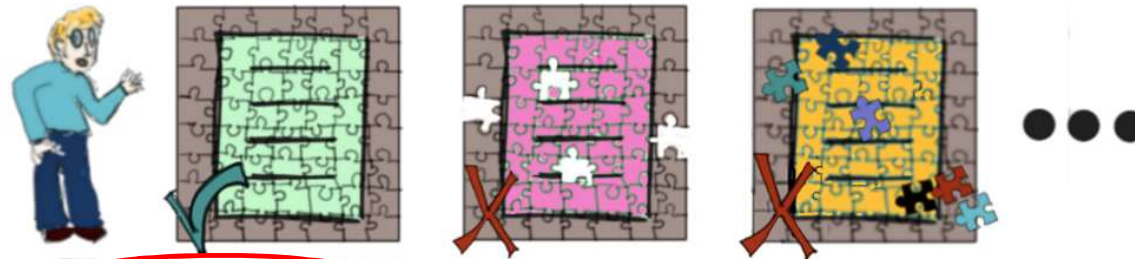
- **FR-1:** The system shall allow users to search for professors by name and department.
- **FR-2:** The system shall display a message when no professors match the search criteria.
- **FR-3:** The system shall allow an administrator to add, update, and remove professor information.
- **NFR-1 — Performance:** The system shall display search results within 2 seconds for 95% of search requests.
- **NFR-2 — Usability:** At least 90% of first-time users shall successfully locate a specified professor within 60 seconds without assistance.
- **NFR-3 — Availability:** The system shall be available 99.5% of the time during normal university operating hours.
- **NFR-4 — Security:** The system shall require administrators to authenticate before adding/removing/modifying professor information.
- **NFR-5 — Compatibility:** The application shall support the two most recent major versions of Chrome, Firefox, Safari, and Edge.
- **NFR-6 — Capacity:** The system shall support at least 500 concurrent users without exceeding a 3-second response time.

Requirements Validation



Verification

*against knowledge
of the dev team*



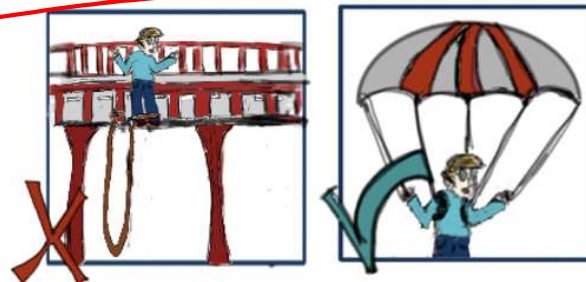
Validation

against stakeholder



Risk Analysis

Spiral



Requirements Engineering Process

1. Requirements Elicitation

1. Consider functional and non-functional requirements

2. Requirements Analysis

1. User research
2. Pertinence and Completeness
3. Prioritize and Categorize

3. Requirements Specification

1. User and system requirements

4. Requirements Validation

1. Verification, Validation, Risk analysis

5. Management

Requirements Engineering Process

